6CCSAPRA

Principles of Robot Autonomy

Revision Notes

Academic Year 2025/26 — Semester 2

Compiled on January 28, 2026

These notes are structured to help you prepare for exams.
Each week includes: learning objectives, core concepts, theory, and worked examples.

Contents

Module Overview **4**

1 Week 1: Introduction to Autonomous Robots, Simple Navigation & Geometry Representation **6**

1.1 Topic Summary . . . 6
1.2 Core Concepts . . . 6
1.2.1 What is an Autonomous Robot? . . . 6
1.2.2 Motion Planning: The Piano Mover's Problem . . . 7
1.3 Bug Algorithms . . . 7
1.3.1 Bug 0 Algorithm . . . 8
1.3.2 Bug 1 Algorithm . . . 8
1.3.3 Bug 2 Algorithm . . . 9
1.3.4 Tangent Bug Algorithm . . . 10
1.4 Geometry Representation . . . 11
1.4.1 Polygons and Polyhedra . . . 11
1.4.2 Convexity . . . 11
1.4.3 Primitives and Half-Spaces . . . 11
1.4.4 Semi-Algebraic Sets . . . 12
1.4.5 Point Clouds and Octomaps . . . 12
1.5 Summary Table: Geometry Representations . . . 13
1.6 Worked Examples . . . 13

2 Week 2: Representations, Transformations & Kinematics **15**

2.1 Topic Summary . . . 15
2.2 Core Concepts . . . 15
2.2.1 Rigid Body Transformations . . . 15
2.2.2 2D Translation . . . 16
2.2.3 2D Rotation . . . 16
2.2.4 Complex Numbers for 2D Rotations . . . 16
2.2.5 Homogeneous Coordinates . . . 17
2.2.6 Homogeneous Transformation Matrices (2D) . . . 17
2.2.7 Rotation About an Arbitrary Point (2D) . . . 18
2.3 3D Transformations . . . 18
2.3.1 3D Rotation Matrices . . . 18
2.3.2 Euler Angles . . . 19
2.3.3 Properties of 3D Rotation Matrices . . . 19
2.3.4 Axis-Angle Representation . . . 19
2.3.5 Gimbal Lock . . . 20
2.3.6 Quaternions . . . 20
2.3.7 3D Homogeneous Transformations . . . 21

2.4 Kinematics 21
2.4.1 Forward Kinematics 21
2.4.2 Inverse Kinematics 22
2.4.3 The Manipulator Jacobian 23
2.5 Summary Tables 24
2.6 Worked Examples 24

Module Overview

About This Module

This module provides an overview of the various challenges that underpin **robot autonomy** and **AI-enabled robotics**. It covers the distinct technical problems involved in allowing robots to perform tasks autonomously and provides students with the algorithms used to solve them.

Module code: 6CCSAPRA **Semester:** 2 (AY 2025/26)

Module leaders: Dr. Khen Elimelech & Dr. Matteo Leonetti

Module Aims

The module aims to:

- Introduce **robot kinematics**—the fundamental language for modelling robot movement.
- Cover **motion planning**—the algorithmic framework enabling complex robots to move to desired positions, even in challenging environments.
- Examine **object grasping** and **task and motion planning (TAMP)** for interactive, complex tasks.
- Explore **localization and mapping**, **handling uncertainty**, and how to leverage **machine learning** in robotics.

Learning Outcomes

By the end of this module, you will be able to:

Code	Learning Outcome
PRA1	**Identify** the main problems of interest at the intersection of AI, ML, and Robotics that underpin robot autonomy, such as planning, perception, state estimation, and learning.
PRA2	**Apply, compare, and analyse** solution methods and algorithms for those problems.
PRA3	**Design and integrate** the algorithmic components needed to create an autonomous system capable of performing a given task.

Key Topics Overview

The module is structured around the following core areas:

Topic	Description
Robot Kinematics	Mathematical models describing robot motion; forward/inverse kinematics; degrees of freedom.
Motion Planning	Path and trajectory planning; sampling-based algorithms (RRT, PRM); configuration space.
Object Grasping	Grasp planning; force closure; manipulation strategies.
Task & Motion Planning	Combining symbolic task planning with geometric motion planning.
Localization & Mapping	SLAM; probabilistic state estimation; sensor fusion.
Handling Uncertainty	Probabilistic robotics; belief states; Bayesian filtering.
Machine Learning	Learning from demonstration; reinforcement learning for robotics.

What Examiners Like to Ask

Based on the learning outcomes, expect questions such as:

- **Conceptual:** "Explain the difference between forward and inverse kinematics."
- **Algorithmic:** "Describe how RRT works and analyse its completeness properties."
- **Comparative:** "Compare PRM and RRT for motion planning."
- **Applied:** "Given a robot arm configuration, compute the end-effector position."
- **Design:** "Design an autonomous system for a pick-and-place task."

[TODO] — Add specific exam question patterns once past papers are available.

1 Week 1: Introduction to Autonomous Robots, Simple Navigation & Geometry Representation

1.1 Topic Summary

Learning Objectives

By the end of this week, you should be able to:

- Explain the core challenges in building autonomous robots (sense–plan–act cycle).
- Describe and analyse Bug algorithms (Bug 0, Bug 1, Bug 2, Tangent Bug).
- Prove completeness and derive path length bounds for Bug algorithms.
- Represent workspace geometry using polygons, primitives, and point clouds.
- Distinguish between convex and non-convex sets and their representations.

Big Picture. This week provides the foundations for robot autonomy. We start with what makes robotics hard (uncertainty, limited sensing, computational cost), then introduce Bug algorithms—simple reactive navigation strategies with provable guarantees. Finally, we cover how to mathematically represent the robot's environment.

What Examiners Like to Ask:

- "Trace Bug 1/Bug 2 on this environment and identify hit/leave points."
- "Derive upper/lower bounds on path length for Bug 1 or Bug 2."
- "Prove that Bug 1 is complete."
- "Compare Bug 1 vs Bug 2—when does each perform better?"
- "Represent this shape using half-planes (primitives)."
- "Define convexity and give an example of a convex hull."

1.2 Core Concepts

Roadmap:

1. What is a robot? The sense–plan–act cycle
2. Bug algorithms for point robot navigation
3. Geometry representation methods

1.2.1 What is an Autonomous Robot?

Robot

A ***robot*** is a physical system that **observes** the state of the world and **changes** the state of the world.

Autonomous Robot

An ***autonomous robot*** performs tasks without continuous human intervention, using its own sensing, planning, and actuation.

The Sense–Plan–Act Cycle:

Component	Description
Sense	Perceive the world using sensors (cameras, LIDAR, tactile sensors).
Plan	Decide what actions to take to achieve a goal.
Act	Execute actions using actuators (motors, grippers).

Key Challenges in Robotics:

- **Limited knowledge:** Sensors are noisy, incomplete, and have finite range.
- **Computational cost:** Planning can be exponentially hard.
- **Hardware limitations:** Control is imperfect; actuators have uncertainty.

1.2.2 Motion Planning: The Piano Mover's Problem

Motion Planning Problem

Given a robot, a start configuration, a goal configuration, and a description of obstacles, find a **collision-free path** from start to goal (or report that none exists).

Exam Tip: Complexity

Motion planning is computationally hard in general:

- Sofa Mover (3 DOF): $O(n^{2+\epsilon})$
- Piano Mover (6 DOF): Polynomial but no practical algorithm known
- n Disks in Plane: NP-hard
- Generalized Mover: PSPACE-complete

This motivates the use of heuristic and sampling-based algorithms.

1.3 Bug Algorithms

Bug algorithms are simple, reactive navigation strategies for a **point robot** with:

- Known direction to goal (can measure distance $d(x, y)$ between any two points).
- Local sensing only (tactile/contact sensor or finite-range sensor).
- No global map of obstacles.

Workspace Notation

- $\mathcal{W} \subseteq \mathbb{R}^2$: the ***workspace*** (environment).
- $\mathcal{WO}_i$: obstacle i in the workspace.
- $\mathcal{W}_{\text{free}} = \mathcal{W} \setminus \bigcup_i \mathcal{WO}_i$: the ***free workspace***.

Assumptions for Bug algorithms:

1. Perfect localization (robot knows its position at all times).
2. Contact sensor available.
3. Robot can measure distance to goal.

4. Finitely many obstacles; a line intersects each obstacle finitely many times.
5. Workspace is bounded: $\mathcal{W} \subset B_r(x)$ for some finite r.

1.3.1 Bug 0 Algorithm

Bug 0 Strategy

1. Head directly toward the goal.
2. If an obstacle is encountered, follow its boundary (fixed direction: left or right) until you can head toward the goal again.
3. Repeat.

Bug 0 Failure Case

Bug 0 is **not complete**. It can get trapped in certain environments.

Example: A left-turning robot facing a "spiral" obstacle that always redirects it away from the goal.

1.3.2 Bug 1 Algorithm

Bug 1 adds **memory** to avoid the traps that defeat Bug 0.

Bug 1 Strategy

1. From current position, head toward q_{goal}.
2. If obstacle encountered at ***hit point*** q_H^i:
   (a) **Circumnavigate** the entire obstacle boundary.
   (b) Record the point q_L^i (***leave point***) closest to the goal.
   (c) Return to q_L^i by wall-following.
3. From q_L^i, head toward goal. If this moves into the obstacle, report **failure**.
4. Repeat until goal reached or failure.

Hit Point and Leave Point

- q_H^i: the ***hit point***—where the robot first contacts obstacle i.
- q_L^i: the ***leave point***—the point on obstacle i's boundary closest to the goal.

Bug 1 Path Length Bounds:
Let D = straight-line distance from q_{start} to q_{goal}, and P_i = perimeter of obstacle i.

Bound	**Expression**
Lower bound	D
Upper bound	$D + \frac{3}{2}\sum_i P_i$

Why $\frac{3}{2}$? In the worst case, the robot circumnavigates the full obstacle (P_i), then backtracks up to half the perimeter to reach the leave point ($\frac{1}{2}P_i$).

Bug 1 Completeness

Theorem: Bug 1 is ***complete***—it will find a path in finite time if one exists, or correctly report failure.

Proof Sketch: Bug 1 Completeness

Suppose Bug 1 is incomplete. Then either:

1. **Never terminates:** But each leave point is strictly closer to the goal than its corresponding hit point. Since there are finitely many obstacles, there are finitely many hit/leave pairs. Eventually, the robot reaches the goal.
2. **Terminates incorrectly:** The leave point q_L^i is the closest point on the boundary to the goal. If moving toward the goal from q_L^i re-enters the obstacle, then by the **Jordan Curve Theorem**, the line to the goal must cross the boundary an even number of times, implying another boundary point is closer—contradicting the definition of q_L^i.

1.3.3 Bug 2 Algorithm

Bug 2 is a **greedy** algorithm that uses the ***m-line*** (the line from q_{start} to q_{goal}).

Bug 2 Strategy

1. Head toward goal along the ***m-line***.
2. If obstacle encountered at hit point q_H^i:
   (a) Follow the obstacle boundary.
   (b) Leave when you re-encounter the m-line at a point **closer to the goal** than q_H^i (and the path to goal is unimpeded).
3. If you return to q_H^i without finding such a point, report **failure**.
4. Repeat until goal reached.

M-Line

The ***m-line*** is the straight line segment connecting q_{start} to q_{goal}.

Bug 2 Path Length Bounds:

Let n_i = number of times the m-line intersects obstacle i.

Bound	**Expression**
Lower bound	D
Upper bound	$D + \frac{1}{2}\sum_i n_i P_i$

Bug 1 vs Bug 2 Comparison

	Bug 1	**Bug 2**
Strategy	Exhaustive (explore all before deciding)	Greedy (take first improvement)
Upper bound	$D + \frac{3}{2}\sum P_i$	$D + \frac{1}{2}\sum n_i P_i$
Performance	More predictable	Often better, but can be worse

Bug 2 beats Bug 1: When obstacles have few m-line crossings.

Bug 1 beats Bug 2: When obstacles have many m-line crossings (e.g., spiral shapes).

1.3.4 Tangent Bug Algorithm

Tangent Bug extends Bug 2 to robots with **finite-range sensors** (not just contact).

Raw Distance Function

The ***raw distance function*** $\rho : \mathbb{R}^2 \times S^1 \to \mathbb{R}$ gives the distance to the closest obstacle along a ray from position x at angle θ:

$$\rho(x, \theta) = \min_{\lambda \in (0,\infty)} d\left(x, x + \lambda(\cos\theta, \sin\theta)^T\right)$$

such that $x + \lambda(\cos\theta, \sin\theta)^T \in \bigcup_i \mathcal{WO}_i$.

The ***saturated*** version $\rho_R(x, \theta)$ clips at sensor range R:

$$\rho_R(x, \theta) = \begin{cases} \rho(x, \theta) & \text{if } \rho(x, \theta) < R \\ \infty & \text{otherwise} \end{cases}$$

Tangent Bug Strategy

Motion-to-goal:

- Move toward goal if path is clear.
- If obstacles visible, choose discontinuity point O_i that minimizes heuristic: $d(x, O_i) + d(O_i, q_{\text{goal}})$.
- Switch to boundary-following if heuristic distance starts increasing.

Boundary-following:

- Maintain d_{followed}: shortest distance from sensed boundary to goal.
- Maintain d_{reach}: shortest distance from blocking obstacle to goal.
- Switch back to motion-to-goal when $d_{\text{reach}} < d_{\text{followed}}$.

Tangent Bug: Effect of Sensor Range

- **Zero range** ($R = 0$): Degenerates to Bug 2 behavior.
- **Finite range**: Can "look ahead" and make smarter decisions.
- **Infinite range**: Optimal path planning (full visibility).

1.4 Geometry Representation

To plan motion, we need to represent:

- The **robot**: set of controllable rigid bodies.
- The **obstacles**: fixed rigid bodies the robot cannot collide with.

1.4.1 Polygons and Polyhedra

Polygon / Polyhedron

- ***Polygon*** (2D): described by ordered sequence of vertices or line segments.
- ***Polyhedron*** (3D): described by vertices, edges, and faces.

Types: triangles, convex polygons, simple polygons, general polygons.

1.4.2 Convexity

Convex Set

A subset $X \subseteq \mathbb{R}^n$ is ***convex*** iff for any two points $x_1, x_2 \in X$, the entire line segment between them lies in X:

$$\lambda x_1 + (1-\lambda)x_2 \in X \quad \forall x_1, x_2 \in X,\ \forall \lambda \in [0,1]$$

A set that is not convex is called ***non-convex***.

Convex Hull

The ***convex hull*** of a set of points $\{v_1, \ldots, v_m\}$ is the smallest convex set containing all points:

$$X = \mathrm{ConvH}(v_1, v_2, \ldots, v_m)$$

For a polytope in $\mathbb{R}^n$, we need $m \geq n+1$ vertices.

Representing Non-Convex Sets

Non-convex polygons can be represented as a **union of convex polygons**:

$$X = X_1 \cup X_2 \cup \cdots \cup X_n$$

Note: Decomposing a non-convex set into convex pieces is NP-hard in general!

1.4.3 Primitives and Half-Spaces

Primitive

A ***primitive*** H is defined by an implicit function $f : \mathbb{R}^n \to \mathbb{R}$:

$$H = \{x \in \mathbb{R}^n \mid f(x) \leq 0\}$$

- **Linear primitive** (half-plane/half-space): $f(x) = ax + by + c$ (2D).

- **Algebraic primitive**: f is a polynomial (e.g., $f(x, y) = x^2 + y^2 - r^2$).

Convex Polytopes via Half-Spaces

A convex polygon (polyhedron) can be described as the **intersection** of half-planes (half-spaces):

$$X = H_1 \cap H_2 \cap \cdots \cap H_m$$

For a polytope in $\mathbb{R}^n$, we need $m \geq n + 1$ half-spaces.

Worked Example: Hexagon via Half-Planes

To define a regular hexagon centered at origin, use 6 half-planes:

$$H_i : a_i x + b_i y + c_i \leq 0, \quad i = 1, \ldots, 6$$

where each (a_i, b_i) is the outward normal to edge i.

1.4.4 Semi-Algebraic Sets

Semi-Algebraic Set

A ***semi-algebraic set*** is constructed using finite unions and intersections of algebraic primitives (sets defined by polynomial inequalities).

Example: Smiley Face

Define a smiley using:

$$\begin{aligned}
H_{\text{head}} &: x^2 + y^2 - r_1^2 \leq 0 \quad \text{(head circle)} \\
H_{\text{Leye}} &: -(x - x_2)^2 - (y - y_2)^2 + r_2^2 \leq 0 \quad \text{(left eye hole)} \\
H_{\text{Reye}} &: -(x - x_3)^2 - (y - y_3)^2 + r_3^2 \leq 0 \quad \text{(right eye hole)} \\
H_{\text{mouth}} &: -\frac{x^2}{a^2} - \frac{(y - y_4)^2}{b^2} + 1 \leq 0 \quad \text{(mouth ellipse)}
\end{aligned}$$

The smiley is: $X = H_{\text{head}} \cap H_{\text{Leye}} \cap H_{\text{Reye}} \cap H_{\text{mouth}}$

1.4.5 Point Clouds and Octomaps

Point Cloud

A ***point cloud*** is a set of 3D points, typically acquired from sensors (LIDAR, depth cameras). Each point represents a surface sample.

Voxel

A ***voxel*** (volumetric pixel) is a 3D grid cell used in occupancy grids.

Octomap

An ***Octomap*** is a probabilistic 3D occupancy map based on ***octrees***:

- Efficiently represents free, occupied, and **unknown** space.
- Supports multi-resolution representation.
- Can fuse multiple sensor readings probabilistically.

Why Octomaps?

- **Memory efficient:** Hierarchical structure; empty/uniform regions use little memory.
- **Handles uncertainty:** Probabilistic occupancy values.
- **Updatable:** New sensor data can be fused incrementally.

1.5 Summary Table: Geometry Representations

Representation	Description	Pros	Cons
Boundary points	Vertices defining polygon	Simple, exact	Limited to polygons
Convex hull	Smallest convex set containing points	Easy intersection tests	Only convex shapes
Half-spaces	Intersection of linear inequalities	Exact, good for collision	Only convex polytopes
Meshes	Triangulated surfaces	Flexible, standard format	No volume info
Algebraic sets	Polynomial inequalities	Smooth curves/surfaces	Complex computations
Point clouds	Raw sensor samples	Direct from sensors	Unstructured, noisy
Octomaps	Hierarchical voxel grid	Efficient, probabilistic	Discrete resolution

---

1.6 Worked Examples

Example 1: Bug 1 Path Analysis

Problem: A point robot uses Bug 1 to navigate from q_{start} to q_{goal} with straight-line distance $D = 10$ units. There are two obstacles with perimeters $P_1 = 6$ and $P_2 = 8$. What are the path length bounds?

Solution:

- Lower bound: $D = 10$ (direct path if no obstacles block).
- Upper bound: $D + \frac{3}{2}(P_1 + P_2) = 10 + \frac{3}{2}(6 + 8) = 10 + 21 = 31$ units.

Example 2: Bug 2 vs Bug 1

Problem: An obstacle has perimeter $P = 20$ and is crossed by the m-line $n = 8$ times. Compare worst-case path contribution for Bug 1 vs Bug 2.

Solution:

- Bug 1 contribution: $\frac{3}{2} \times 20 = 30$ units.

- Bug 2 contribution: $\frac{1}{2} \times 8 \times 20 = 80$ units.

Bug 1 performs better here because the obstacle has many m-line crossings.

Example 3: Convexity Check

Problem: Is the set $X = \{(x, y) : x^2 + y^2 \leq 1\}$ convex?

Solution: Yes. For any two points p_1, p_2 on or inside the unit circle, the line segment between them lies entirely within the circle. This follows from the triangle inequality.

Example 4: Triangle via Half-Planes

Problem: Represent the triangle with vertices $(0, 0)$, $(4, 0)$, $(2, 3)$ using half-planes.

Solution: Find the three edge equations:

- Edge 1 ($(0, 0)$ to $(4, 0)$): $y \geq 0 \Rightarrow -y \leq 0$
- Edge 2 ($(4, 0)$ to $(2, 3)$): $3x + 2y \leq 12$
- Edge 3 ($(2, 3)$ to $(0, 0)$): $3x - 2y \geq 0 \Rightarrow -3x + 2y \leq 0$

Triangle: $H_1 \cap H_2 \cap H_3$.

2 Week 2: Representations, Transformations & Kinematics

2.1 Topic Summary

Learning Objectives

By the end of this week, you should be able to:

- Apply 2D and 3D rigid body transformations (translation, rotation).
- Construct and multiply homogeneous transformation matrices.
- Explain Euler angles, gimbal lock, and quaternions for 3D rotations.
- Compute forward kinematics for kinematic chains.
- Set up and solve inverse kinematics problems analytically.
- Explain the role of the Jacobian in numerical inverse kinematics.

Big Picture. To plan robot motion, we need to mathematically describe *where* things are. This week covers rigid body transformations—the mathematical tools for specifying position and orientation in 2D and 3D. We then extend to kinematic chains (robot arms) and the fundamental problems of forward and inverse kinematics.

What Examiners Like to Ask:

- "Compute the homogeneous transformation matrix for rotation θ then translation t."
- "Apply a sequence of transformations to find the final pose."
- "Explain gimbal lock and how quaternions avoid it."
- "Derive forward kinematics for a 2-link planar arm."
- "Solve inverse kinematics for a 2-link arm given end-effector position."
- "What is the Jacobian and how is it used in IK?"

2.2 Core Concepts

Roadmap:

1. Rigid body transformations in 2D
2. Homogeneous coordinates and transformation matrices
3. 3D rotations: Euler angles, axis-angle, quaternions
4. Kinematic chains and forward kinematics
5. Inverse kinematics

2.2.1 Rigid Body Transformations

Rigid Body Transformation

A ***rigid body transformation*** $h : A \to W$ maps every point of body A into workspace W while:

1. Preserving distances between all pairs of points.
2. Preserving orientation (no mirror images).

A rigid body in 2D is fully described by:

- **Position:** $(x, y) \in \mathbb{R}^2$

- **Orientation:** angle $\theta \in [0, 2\pi)$

2.2.2 2D Translation

Translation in 2D

Translation by (x_t, y_t):

$$h(x, y) = (x + x_t, y + y_t)$$

For primitives $H_i = \{(x, y) : f(x, y) \leq 0\}$:

$$h(H_i) = \{(x, y) \in W : f(x - x_t, y - y_t) \leq 0\}$$

2.2.3 2D Rotation

Rotation Matrix in 2D

Rotation about the origin by angle θ (counter-clockwise):

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

A point $v = (x, y)^T$ transforms to:

$$v' = R(\theta) \cdot v$$

Properties of 2D Rotation Matrices

- **Orthogonal:** $R^T R = RR^T = I$
- **Determinant:** $\det(R) = 1$
- **Inverse:** $R(\theta)^{-1} = R(-\theta) = R(\theta)^T$
- **Composition:** $R(\theta_2)R(\theta_1) = R(\theta_1 + \theta_2)$
- **2D rotations commute:** $R(\theta_1)R(\theta_2) = R(\theta_2)R(\theta_1)$

Right-Hand Rule

Positive rotation is **counter-clockwise** (direction fingers curl when thumb points along rotation axis). In 2D, rotations are about the z-axis extending "out of the page."

2.2.4 Complex Numbers for 2D Rotations

Rotation via Complex Multiplication

Complex numbers with unit norm can represent 2D rotations:

$$z = \cos\theta + i\sin\theta = e^{i\theta}$$

Rotating point (x, y) by θ: multiply complex number $x + iy$ by $e^{i\theta}$.

2.2.5 Homogeneous Coordinates

Homogeneous Coordinates

A 2D point (x, y) is represented in ***homogeneous coordinates*** as $(x, y, 1)^T$.
More generally, (x, y, w) represents the Cartesian point $(x/w, y/w)$ for $w \neq 0$.
Points at infinity: $(x, y, 0)$ represents infinity in direction (x, y).

Why homogeneous coordinates?

- Combine rotation and translation into a single matrix multiplication.
- Represent points at infinity with finite coordinates.
- Simplify composition of multiple transformations.

2.2.6 Homogeneous Transformation Matrices (2D)

2D Homogeneous Transformation

A rigid body transformation (rotation θ followed by translation $t = (x_t, y_t)^T$):

$$T = \begin{pmatrix} R(\theta) & t \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta & x_t \\ \sin\theta & \cos\theta & y_t \\ 0 & 0 & 1 \end{pmatrix}$$

Applied to point v:

$$\begin{pmatrix} v' \\ 1 \end{pmatrix} = T \begin{pmatrix} v \\ 1 \end{pmatrix} = \begin{pmatrix} R(\theta)v + t \\ 1 \end{pmatrix}$$

Structure of Homogeneous Transform

$$T = \begin{pmatrix} \text{Rotation} & \text{Translation} \\ \text{Perspective} & \text{Scale} \end{pmatrix} = \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix}$$

For rigid body transforms: perspective $= 0$, scale $= 1$.

Successive Transformations

Sequence: Rotate by $\theta_1 \to$ Translate by $t_1 \to$ Rotate by $\theta_2 \to$ Translate by t_2
Combined matrix:

$$T_{1,2} = \begin{pmatrix} R(\theta_2) & t_2 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} R(\theta_1) & t_1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} R(\theta_2)R(\theta_1) & R(\theta_2)t_1 + t_2 \\ 0 & 1 \end{pmatrix}$$

Key insight: Transformations are applied right-to-left (matrix on the right acts first).

2.2.7 Rotation About an Arbitrary Point (2D)

Rotation About Point p

To rotate about point p (not the origin):

1. Translate by $-p$ (move p to origin)
2. Rotate by θ
3. Translate by $+p$ (move back)

$$T = \begin{pmatrix} I & p \\ 0 & 1 \end{pmatrix} \begin{pmatrix} R(\theta) & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} I & -p \\ 0 & 1 \end{pmatrix}$$

2.3 3D Transformations

A rigid body in 3D requires:

- **Position:** $(x, y, z) \in \mathbb{R}^3$ (3 parameters)
- **Orientation:** 3 parameters (various representations)

2.3.1 3D Rotation Matrices

Elementary Rotation Matrices

Rotations about the coordinate axes (right-hand rule, counter-clockwise):

Roll (about x-axis):

$$R_x(\gamma) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\gamma & -\sin\gamma \\ 0 & \sin\gamma & \cos\gamma \end{pmatrix}$$

Pitch (about y-axis):

$$R_y(\beta) = \begin{pmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{pmatrix}$$

Yaw (about z-axis):

$$R_z(\alpha) = \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Exam Tip: Memory Aid

Each rotation matrix has 1 on the diagonal for the axis of rotation (that coordinate is unchanged). The 2×2 block in the other positions is the standard 2D rotation matrix.

2.3.2 Euler Angles

Euler Angles

Any 3D orientation can be achieved by **three successive rotations** about coordinate axes, where no two successive rotations are about the same axis.

Common convention (ZYX / yaw-pitch-roll):

$$R(\alpha, \beta, \gamma) = R_z(\alpha)R_y(\beta)R_x(\gamma)$$

Read right-to-left: roll γ about x, then pitch β about y, then yaw α about z.

Valid Euler angle sequences:

- Proper Euler: $XYX, XZX, YXY, YZY, ZXZ, ZYZ$
- Tait-Bryan: $XYZ, XZY, YXZ, YZX, ZXY, ZYX$ (includes yaw-pitch-roll)

3D Rotations Do NOT Commute

Unlike 2D, 3D rotations do not commute:

$$R_z\left(\frac{\pi}{2}\right)R_y\left(\frac{\pi}{2}\right) \neq R_y\left(\frac{\pi}{2}\right)R_z\left(\frac{\pi}{2}\right)$$

Order matters!

2.3.3 Properties of 3D Rotation Matrices

Special Orthogonal Group $SO(3)$

3D rotation matrices belong to $SO(3)$:

$$SO(3) = \{R \in \mathbb{R}^{3\times 3} : RR^T = I \text{ and } \det(R) = 1\}$$

- Columns are mutually orthonormal: $r_i^T r_j = \delta_{ij}$
- Inverse: $R^{-1} = R^T$
- $\det(R) = +1$ (not -1, which would be a reflection)

2.3.4 Axis-Angle Representation

Rotation About Arbitrary Axis

Any 3D rotation can be described by:

- Unit vector $\mathbf{k} = (k_x, k_y, k_z)$ along the rotation axis
- Angle θ of rotation about that axis

The rotation matrix is computed by transforming to align with a coordinate axis:

$$R_{\mathbf{k}}(\theta) = R_z(\alpha)R_y(\beta)R_z(\theta)R_y(-\beta)R_z(-\alpha)$$

where α, β align $\mathbf{k}$ with the z-axis.

2.3.5 Gimbal Lock

Gimbal Lock

Gimbal lock occurs when two rotation axes become aligned, causing **loss of one degree of freedom**.

Example: When pitch $\beta = \pm 90°$, yaw and roll rotations produce the same effect.

Gimbal Lock Problem

- Occurs with *all* Euler angle representations (just at different configurations).
- Historical example: Apollo missions had to avoid certain orientations.
- Solution: Use quaternions!

2.3.6 Quaternions

Quaternion

A ***quaternion*** is a 4-dimensional extension of complex numbers:

$$q = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k} = (a, \mathbf{v}) \quad \text{where } \mathbf{v} = (b, c, d)$$

With multiplication rules:

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = -1, \quad \mathbf{ij} = \mathbf{k}, \quad \mathbf{jk} = \mathbf{i}, \quad \mathbf{ki} = \mathbf{j}$$

Unit Quaternion for Rotation

A ***unit quaternion*** ($\|q\| = 1$) represents rotation by angle θ about unit axis $\mathbf{n}$:

$$q = \cos\frac{\theta}{2} + \sin\frac{\theta}{2}(n_x\mathbf{i} + n_y\mathbf{j} + n_z\mathbf{k}) = e^{\frac{\theta}{2}\mathbf{n}}$$

Rotating a Point with Quaternions

To rotate point (x, y, z) by quaternion q:

1. Represent point as pure quaternion: $v = x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$
2. Compute: $v' = q \cdot v \cdot q^*$ (quaternion "sandwich")
3. Extract rotated point from $v' = x'\mathbf{i} + y'\mathbf{j} + z'\mathbf{k}$

where $q^* = a - b\mathbf{i} - c\mathbf{j} - d\mathbf{k}$ is the conjugate.

Advantages of Quaternions

Property	**Benefit**
Compact	4 parameters vs. 9 for rotation matrix
No gimbal lock	Avoids singularities entirely
Easy renormalization	Just normalize to unit length
Stable interpolation	SLERP gives smooth rotation interpolation
Numerically stable	Avoids accumulating errors from matrix multiplication

Disadvantage: Not intuitive; must convert to matrix to visualize.

SLERP: Spherical Linear Interpolation

Interpolate between quaternions q_1 and q_2:

$$\text{SLERP}(q_1, q_2, t) = \frac{\sin((1-t)\phi)}{\sin\phi} q_1 + \frac{\sin(t\phi)}{\sin\phi} q_2$$

where ϕ is the angle between q_1 and q_2. Always produces a unit quaternion.

2.3.7 3D Homogeneous Transformations

3D Homogeneous Transformation Matrix

Rotation R followed by translation $t = (x_t, y_t, z_t)^T$:

$$T = \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix} \in \mathbb{R}^{4\times 4}$$

Applied to point v:

$$\begin{pmatrix} v' \\ 1 \end{pmatrix} = T \begin{pmatrix} v \\ 1 \end{pmatrix}$$

2.4 Kinematics

Kinematics

Kinematics is the study of motion and configurations of a system—the "geometry of the system"—without considering forces.

Key concepts:

- ***Link***: A rigid body in a chain of rigid bodies.
- ***Joint***: Connects two links and constrains their relative motion.
- ***End-effector***: The "hand" or tool at the end of the kinematic chain.

2.4.1 Forward Kinematics

Forward Kinematics (FK)

Given joint angles/positions $\mathbf{q} = (q_1, \ldots, q_n)$, compute the position and orientation of the end-effector in the world frame:

$$\mathbf{x} = FK(\mathbf{q})$$

FK via Transformation Matrices

For a chain of n links, each with transformation T_i from link i to link $i-1$:

$$T_{\text{total}} = T_1 T_2 \cdots T_n T_{\text{ee}}$$

where T_{ee} is the end-effector offset from the last joint.
The end-effector position in the world frame:

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = T_{\text{total}} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

2D 3-Link Planar Arm

For a 3-link arm with link lengths a_1, a_2, a_3 and joint angles $\theta_1, \theta_2, \theta_3$:
Transformation matrices:

$$T_i = \begin{pmatrix} \cos\theta_i & -\sin\theta_i & a_{i-1} \\ \sin\theta_i & \cos\theta_i & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

End-effector position:

$$\begin{aligned} x &= a_1\cos\theta_1 + a_2\cos(\theta_1+\theta_2) + a_3\cos(\theta_1+\theta_2+\theta_3) \\ y &= a_1\sin\theta_1 + a_2\sin(\theta_1+\theta_2) + a_3\sin(\theta_1+\theta_2+\theta_3) \end{aligned}$$

2.4.2 Inverse Kinematics

Inverse Kinematics (IK)

Given a desired end-effector position/orientation $\mathbf{x}_{\text{target}}$, find joint angles $\mathbf{q}$ such that:

$$FK(\mathbf{q}) = \mathbf{x}_{\text{target}}$$

IK Solution Cases

Let n = number of joint DOFs, m = number of task DOFs.

Condition	Solutions
$n < m$	No solution (overconstrained)
$n = m$	Finite number of solutions (if reachable)
$n > m$	Infinite solutions (robot is ***redundant***)

Analytic IK for 2-Link Planar Arm

Given: Target (x, y), link lengths a_1, a_2.
Forward kinematics:

$$\begin{aligned} x &= a_1\cos\theta_1 + a_2\cos(\theta_1+\theta_2) \\ y &= a_1\sin\theta_1 + a_2\sin(\theta_1+\theta_2) \end{aligned}$$

Solution for θ_2: Square and add the FK equations:

$$x^2 + y^2 = a_1^2 + a_2^2 + 2a_1a_2\cos\theta_2$$

$$\cos\theta_2 = \frac{x^2 + y^2 - a_1^2 - a_2^2}{2a_1a_2}$$
$$\sin\theta_2 = \pm\sqrt{1 - \cos^2\theta_2}$$

Solution for θ_1: Substitute back:

$$\cos\theta_1 = \frac{1}{x^2 + y^2}\left[x(a_1 + a_2\cos\theta_2) \pm y \cdot a_2\sin\theta_2\right]$$
$$\sin\theta_1 = \frac{1}{x^2 + y^2}\left[y(a_1 + a_2\cos\theta_2) \mp x \cdot a_2\sin\theta_2\right]$$

Two solutions: "Elbow up" and "elbow down" configurations.

Singularity

A ***singularity*** occurs when the robot loses one or more degrees of freedom—only one IK solution exists instead of two (or the Jacobian loses rank).

Examples:

- Arm fully extended: $\theta_2 = 0$
- Arm fully folded: $\theta_2 = \pi$

2.4.3 The Manipulator Jacobian

Jacobian Matrix

The ***Jacobian*** J relates joint velocities to end-effector velocities:

$$\dot{\mathbf{x}} = J(\mathbf{q}) \cdot \dot{\mathbf{q}}$$

where:

$$J = \frac{\partial FK}{\partial \mathbf{q}} = \begin{pmatrix} \frac{\partial x}{\partial q_1} & \cdots & \frac{\partial x}{\partial q_n} \\ \frac{\partial y}{\partial q_1} & \cdots & \frac{\partial y}{\partial q_n} \\ \vdots & & \vdots \end{pmatrix}$$

Numerical IK via Jacobian

To solve IK iteratively:

$$\Delta\mathbf{q} = J^{-1}\Delta\mathbf{x}$$

where $\Delta\mathbf{x} = \mathbf{x}_{\text{target}} - FK(\mathbf{q})$.

Gradient descent approach:

1. Compute error: $\Delta\mathbf{x} = \mathbf{x}_{\text{target}} - FK(\mathbf{q})$
2. Compute Jacobian $J(\mathbf{q})$
3. Update: $\mathbf{q} \leftarrow \mathbf{q} + \alpha J^{-1}\Delta\mathbf{x}$
4. Repeat until $\|\Delta\mathbf{x}\| < \epsilon$

For non-square J, use pseudoinverse J^+ or transpose J^T.

Jacobian and Singularities

At a **singularity**, the Jacobian loses rank (rows become linearly dependent). This means:

- Some end-effector velocities are impossible.
- J^{-1} is undefined or numerically unstable.
- Use **damped least squares** to handle near-singularities.

2.5 Summary Tables

Rotation Representation	Parameters	Gimbal Lock?	Interpolation
Rotation matrix	9 (with 6 constraints)	No	Difficult
Euler angles	3	Yes	Linear (discontinuous)
Axis-angle	4 (with 1 constraint)	At $\theta = 0$	Difficult
Quaternion	4 (with 1 constraint)	No	SLERP (smooth)

2.6 Worked Examples

Example 1: 2D Transformation Sequence

Problem: A robot at origin is rotated by 90° then translated by $(3, 2)$. Find the transformation matrix and the final position of point $(1, 0)$.

Solution:

$$T = \begin{pmatrix} \cos 90^\circ & -\sin 90^\circ & 3 \\ \sin 90^\circ & \cos 90^\circ & 2 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 3 \\ 1 & 0 & 2 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 3 \\ 1 & 0 & 2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \\ 1 \end{pmatrix}$$

Final position: $(3, 3)$.

Example 2: 2-Link IK

Problem: A 2-link arm has $a_1 = a_2 = 1$. Find θ_1, θ_2 for end-effector at $(1, 1)$.

Solution:

$$\cos\theta_2 = \frac{1^2 + 1^2 - 1 - 1}{2(1)(1)} = \frac{0}{2} = 0 \implies \theta_2 = \pm 90^\circ$$

For $\theta_2 = 90^\circ$: $\sin\theta_2 = 1$

$$\cos\theta_1 = \frac{1(1+0) + 1(1)}{2} = 1, \quad \sin\theta_1 = \frac{1(1+0) - 1(1)}{2} = 0$$

$\theta_1 = 0^\circ$

Check: $x = \cos 0^\circ + \cos 90^\circ = 1$, $y = \sin 0^\circ + \sin 90^\circ = 1$. ✓

Example 3: Quaternion Rotation

Problem: Express a 180° rotation about the z-axis as a quaternion.

Solution:

$$q = \cos\frac{180^\circ}{2} + \sin\frac{180^\circ}{2}(0\mathbf{i} + 0\mathbf{j} + 1\mathbf{k}) = \cos 90^\circ + \sin 90^\circ \cdot \mathbf{k} = 0 + 1\mathbf{k} = \mathbf{k}$$

Or in (a, b, c, d) form: $q = (0, 0, 0, 1)$.